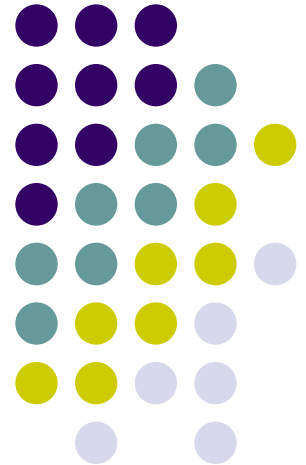


php



Dr:Wafaa Alnbhany

Lecture6



PHP Looping

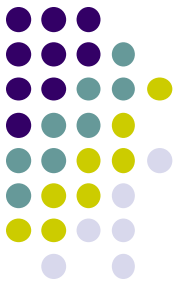
Looping statements in PHP are used to execute the same block of code a specified number of times.

Looping

Very often when you write code, you want the same block of code to run a number of times. You can use looping statements in your code to perform this.

In PHP we have the following looping statements:

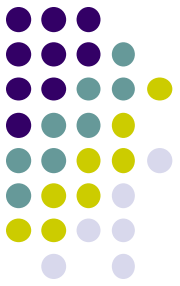
- **while** - loops through a block of code if and as long as a specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as a special condition is true
- **for** - loops through a block of code a specified number of times
- **foreach** - loops through a block of code for each element in an array



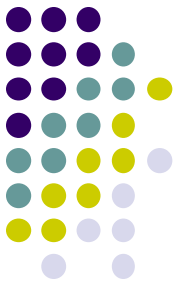
Definition and Usage

- The `printf()` function outputs a formatted string.
- The `arg1`, `arg2`, ++ parameters are inserted at percent (%) signs in the main string.
- At the first % sign, `arg1` is inserted; at the second % sign, `arg2` is inserted, and so on.
- If there are more % signs than arguments, placeholders must be used.
- A placeholder consists of the argument number and the "\$" symbol.

Syntax



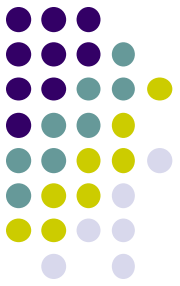
- `printf(format, arg1, arg2, arg3, ...);`



Parameter Values

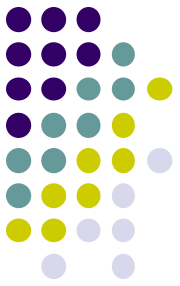
- • format (Required): Specifies the string and how to format the variables.
- Common format specifiers:
 - • %% - Percent sign
 - • %b - Binary number
 - • %c - Character (ASCII)
 - • %d - Signed decimal number
 - • %e / %E - Scientific notation
 - • %u - Unsigned decimal number
 - • %f - Floating-point number
 - • %g / %G - Shorter of %e or %f
 - • %o - Octal number
 - • %s - String

Example 1: Basic printf() Usage



- `<?php`
 - `$num = 10;`
 - `printf('Number: %d', $num);`
 - `?>`
-
- Output:
 - Number: 10

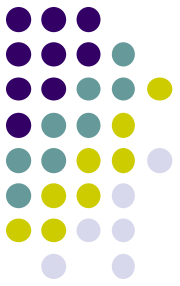
Example 2: Using Placeholders



- `<?php`
- `$a = 'Wafaa';`
- `$b = 'PHP';`
- `printf('%2$s is teaching %1$s', $a, $b);`
- `?>`

- Output:
- PHP is teaching Wafaa

Example 3: Multiple Data Types



- `<?php`
- `$num = 255;`
- `printf('Decimal: %d, Hex: %X, Octal: %o',
$num, $num, $num);`
- `?>`

- Output:
- Decimal: 255, Hex: FF, Octal: 377

PHP Date()

The PHP date() function is used to format a time or a date.

The PHP Date() Function

The PHP date() function formats a timestamp to a more readable date and time.

Syntax

```
date (format, timestamp)
```

Parameter	Description
format	Required. Specifies the format of the timestamp
timestamp	Optional. Specifies a timestamp. Default is the current date and time (as a timestamp)

PHP Date - What is a Timestamp?

A timestamp is the number of seconds since January 1, 1970 at 00:00:00 GMT. This is also known as the Unix Timestamp.

PHP Date - Format the Date

The first parameter in the `date()` function specifies how to format the date/time. It uses letters to represent date and time formats. Here are some of the letters that can be used:

- `d` - The day of the month (01-31)
- `m` - The current month, as a number (01-12)
- `Y` - The current year in four digits

An overview of all the letters that can be used in the format parameter, can be found in our [PHP Date reference](#).

Other characters, like "/", ".", or "-" can also be inserted between the letters to add additional formatting:

```
<?php  
echo date("Y/m/d");  
echo "<br />";  
echo date("Y.m.d");  
echo "<br />";  
echo date("Y-m-d");  
?>
```

The output of the code above could be something like this:

```
2006/07/11  
2006.07.11  
2006-07-11
```

PHP Date - Adding a Timestamp



The second parameter in the `date()` function specifies a timestamp. This parameter is optional. If you do not supply a timestamp, the current time will be used.

In our next example we will use the `mktime()` function to create a timestamp for tomorrow.

The `mktime()` function returns the Unix timestamp for a specified date.

Syntax

```
mktime(hour, minute, second, month, day, year, is_dst)
```

To go one day in the future we simply add one to the day argument of `mktime()`:

```
<?php
$tomorrow = mktime(0,0,0,date("m"),date("d")+1,date("Y"));
echo "Tomorrow is ".date("Y/m/d", $tomorrow);
?>
```

The output of the code above could be something like this:

```
Tomorrow is 2006/07/12
```

PHP File Handling



The `fopen()` function is used to open files in PHP.

Opening a File

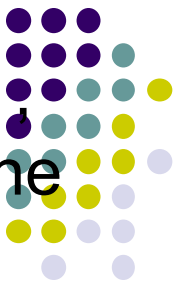
The `fopen()` function is used to open files in PHP.

The first parameter of this function contains the name of the file to be opened and the second parameter specifies in which mode the file should be opened:

```
<html>
<body>
<?php
$file=fopen("welcome.txt","r");
?>
</body>
</html>
```

The file may be opened in one of the following modes:

Modes	Description
r	Read only. Starts at the beginning of the file
r+	Read/Write. Starts at the beginning of the file
w	Write only. Opens and clears the contents of file; or creates a new file if it doesn't exist
w+	Read/Write. Opens and clears the contents of file; or creates a new file if it doesn't exist
a	Append. Opens and writes to the end of the file or creates a new file if it doesn't exist
a+	Read/Append. Preserves file content by writing to the end of the file
x	Write only. Creates a new file. Returns FALSE and an error if file already exists
x+	Read/Write. Creates a new file. Returns FALSE and an error if file already exists



- The W create a new file or truncates an existing file , then opens it for writing, the file pointer position at the beginning of file.
- The W+ create a new file or truncates an existing file , then opens it for reading and writing, the file pointer position at the beginning of file.
- Both r+ and w+ can read and write to a file. However, **r+ doesn't delete the content of the file and doesn't create a new file if such file doesn't exist, whereas w+ deletes the content of the file and creates it if it doesn't exist**
- There is an important difference between a and a+, a append data in a file, it can update the file writing some data at the end, a+ append data in a file, it can update the file writing some data at the end and also is able to read the file

Note: If the `fopen()` function is unable to open the specified file, it returns 0 (false).

Example

The following example generates a message if the `fopen()` function is unable to open the specified file:

```
<html>
<body>
<?php
$file=fopen("welcome.txt","r") or exit("Unable to open
file!");
?>
</body>
</html>
```

Closing a File

The `fclose()` function is used to close an open file:

```
<?php
$file = fopen("test.txt","r");
//some code to be executed
fclose($file);
?>
```

Check End-of-file

The `feof()` function checks if the "end-of-file" (EOF) has been reached. The `feof()` function is useful for looping through data of unknown length.

Note: You cannot read from files opened in `w`, `a`, and `x` mode!

```
if (feof($file)) echo "End of file";
```

Reading a File Line by Line

The `fgets()` function is used to read a single line from a file.

Note: After a call to this function the file pointer has moved to the next line.

Example

The example below reads a file line by line, until the end of file is reached:

```
<?php
$file = fopen("welcome.txt", "r") or exit("Unable to open
file!");
//Output a line of the file until the end is reached
while(!feof($file))
{
    echo fgets($file). "<br />";
}
fclose($file);
?>
```

Reading a File Character by Character

The `fgetc()` function is used to read a single character from a file.

Note: After a call to this function the file pointer moves to the next character.

Example

The example below reads a file character by character, until the end of file is reached:

```
<?php
$file=fopen("welcome.txt","r") or exit("Unable to open
file!");
while (!feof($file))
{
    echo fgetc($file);
}
fclose($file);
?>
```

PHP File Upload

With PHP, it is possible to upload files to the server.

Create an Upload-File Form

To allow users to upload files from a form can be very useful.

Look at the following HTML form for uploading files:

```
<html>
<body>
<form action="upload_file.php" method="post"
enctype="multipart/form-data">
<label for="file">Filename:</label>
<input type="file" name="file" id="file" />
<br />
<input type="submit" name="submit" value="Submit" />
</form>
</body>
</html>
```

Notice the following about the HTML form above:

- The enctype attribute of the <form> tag specifies which content-type to use when submitting the form. "multipart/form-data" is used when a form requires binary data, like the contents of a file, to be uploaded
- The type="file" attribute of the <input> tag specifies that the input should be processed as a file. For example, when viewed in a browser, there will be a browse-button next to the input field

Note: Allowing users to upload files is a big security risk. Only permit trusted users to perform file uploads.

Create The Upload Script

The "upload_file.php" file contains the code for uploading a file:

```
<?php
if ($_FILES["file"]["error"] > 0)
{
    echo "Error: " . $_FILES["file"]["error"] . "<br />";
}
else
{
    echo "Upload: " . $_FILES["file"]["name"] . "<br />";
```

```
    echo "Type: " . $_FILES["file"]["type"] . "<br />";
    echo "Size: " . ($_FILES["file"]["size"] / 1024) . " Kb<br />";
    echo "Stored in: " . $_FILES["file"]["tmp_name"];
}
?>
```

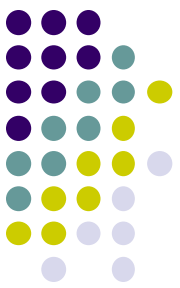
By using the global PHP `$_FILES` array you can upload files from a client computer to the remote server.

The first parameter is the form's input name and the second index can be either "name", "type", "size", "tmp_name" or "error". Like this:

- `$_FILES["file"]["name"]` - the name of the uploaded file
- `$_FILES["file"]["type"]` - the type of the uploaded file
- `$_FILES["file"]["size"]` - the size in bytes of the uploaded file
- `$_FILES["file"]["tmp_name"]` - the name of the temporary copy of the file stored on the server
- `$_FILES["file"]["error"]` - the error code resulting from the file upload

This is a very simple way of uploading files. For security reasons, you should add restrictions on what the user is allowed to upload.

Restrictions on Upload



In this script we add some restrictions to the file upload. The user may only upload .gif or .jpeg files and the file size must be under 20 kb:

```
<?php
if ((($_FILES["file"]["type"] == "image/gif")
|| ($_FILES["file"]["type"] == "image/jpeg")
|| ($_FILES["file"]["type"] == "image/pjpeg"))
&& ($_FILES["file"]["size"] < 20000))
{
    if ($_FILES["file"]["error"] > 0)
    {
        echo "Error: " . $_FILES["file"]["error"] . "<br />";
    }
    else
    {
        echo "Upload: " . $_FILES["file"]["name"] . "<br />";
        echo "Type: " . $_FILES["file"]["type"] . "<br />";
        echo "Size: " . ($_FILES["file"]["size"] / 1024) . " Kb<br />";
        echo "Stored in: " . $_FILES["file"]["tmp_name"];

    }
}
else
{
    echo "Invalid file";
}
?>
```

Saving the Uploaded File

The examples above create a temporary copy of the uploaded files in the PHP temp folder on the server.

The temporary copied files disappears when the script ends. To store the uploaded file we need to copy it to a different location:

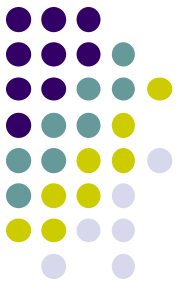
```
<?php
if ((($_FILES["file"]["type"] == "image/gif")
|| ($_FILES["file"]["type"] == "image/jpeg")
|| ($_FILES["file"]["type"] == "image/pjpeg"))
&& ($_FILES["file"]["size"] < 20000))
{
    if ($_FILES["file"]["error"] > 0)
    {
        echo "Return Code: " . $_FILES["file"]["error"] . "<br
/>";
    }
    else
    {
        echo "Upload: " . $_FILES["file"]["name"] . "<br />";
        echo "Type: " . $_FILES["file"]["type"] . "<br />";
        echo "Size: " . ($_FILES["file"]["size"] / 1024) . " Kb<br
/>";
        echo "Temp file: " . $_FILES["file"]["tmp_name"] . "<br
/>";
        if (file_exists("upload/" . $_FILES["file"]["name"]))
        {
            echo $_FILES["file"]["name"] . " already exists. ";
        }
        else
        {
            move_uploaded_file($_FILES["file"]["tmp_name"],
            "upload/" . $_FILES["file"]["name"]);
            echo "Stored in: " . "upload/" .
```

```
$_FILES["file"]["name"];
    }
}
}
else
{
    echo "Invalid file";
}
?>
```

The script above checks if the file already exists, if it does not, it copies the file to the specified folder.

Note: This example saves the file to a new folder called "upload"

H.W



- Write txt file with your name then
 - 1- display the content line by line in php page
 - 2- display the content character by character in php page
 - 3- Ask the user to enter CV , then save CV file on the server